

FRONTEND WEB DESIGN

Introduction to Frontend Web Development

Frontend web development is the practice of building the part of a website or application that users see and interact with directly inside their browser. It combines three core technologies: HTML5 for structure and content, CSS3 for presentation and layout, and JavaScript for behaviour and interactivity. This manual focuses on HTML5 and CSS3 — including Flexbox, Grid and the Tailwind CSS utility framework — along with practical UI component patterns and productivity tips for Visual Studio Code.

How a Web Page is Built

- HTML (HyperText Markup Language) — defines the structure and meaning of content using elements/tags.
- CSS (Cascading Style Sheets) — controls colours, spacing, fonts, layout and responsiveness.
- JavaScript — adds interactivity, logic and dynamic behaviour (not covered in depth in this manual).

Recommended Workflow

- Plan the page structure first using semantic HTML.
- Add a CSS reset / normalize and set box-sizing globally.
- Build layout using Flexbox or Grid before styling details.
- Style typography, colour and spacing consistently using a small design scale.
- Test responsiveness using a mobile-first approach with media queries.

HTML5 — Document Structure

Every HTML5 document follows a standard structure. The table below lists the essential structural elements found in the `<head>` and the root of any HTML document.

Item	Description	Example
<code><!DOCTYPE html></code>	Declares an HTML5 document	<code><!DOCTYPE html></code>
<code><html lang></code>	Root element; sets page language	<code><html lang="en"></code>
<code><head></code>	Container for meta information (not visible)	<code><head>...</head></code>
<code><meta charset></code>	Character encoding of the document	<code><meta charset="UTF-8"></code>
<code><meta viewport></code>	Enables responsive scaling on mobile	<code><meta name="viewport" content="width=device-width, initial-scale=1"></code>
<code><title></code>	Text shown in the browser tab	<code><title>My Page</title></code>
<code><link></code>	Attaches external CSS or favicon	<code><link rel="stylesheet" href="style.css"></code>

Item	Description	Example
<script>	Includes a JavaScript file or inline code	<script src="app.js" defer></script>
<body>	Contains all visible page content	<body>...</body>

Minimal HTML5 Boilerplate

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello World</h1>
  <script src="app.js" defer></script>
</body>
</html>
```

HTML5 — Semantic Layout Elements

Semantic elements clearly describe their purpose to both the browser and other developers. They also improve accessibility and search engine optimisation (SEO), because screen readers and search engines understand the role of each section.

Item	Description	Example
<header>	Intro/banner area, logo, navigation	<header> Site </header>
<nav>	Group of navigation links	<nav>Home</nav>
<main>	Main unique content (once per page)	<main>...</main>
<section>	Thematic grouping of related content	<section> About </section>
<article>	Self-contained, reusable content block	<article>Blog post</article>
<aside>	Side content, related links, ads	<aside>Related links</aside>
<footer>	Footer information, copyright	<footer>© 2026</footer>
<div>	Generic block-level container (no meaning)	<div class="card">...</div>
	Generic inline container (no meaning)	new

Typical Page Skeleton

```
<body>
  <header>
    <nav>
      <a href="#home">Home</a>
      <a href="#about">About</a>
    </nav>
```

```

</header>

<main>
  <section id="about">
    <h2>About Us</h2>
    <p>Some content...</p>
  </section>
  <article>
    <h2>Blog Post Title</h2>
    <p>Article content...</p>
  </article>
</main>

<aside>Related links</aside>

<footer>
  <p>&copy; 2026 My Site</p>
</footer>
</body>

```

HTML5 — Text and Content Elements

Item	Description	Example
<h1> – <h6>	Headings; h1 = highest importance	Title
<p>	Paragraph of text	Text here.
<a href>	Hyperlink to another page or section	Link
	Embeds an image (self-closing)	
	Unordered (bulleted) list	Item
	Ordered (numbered) list	Item
/	Semantic bold / italic emphasis	bold
 / <hr>	Line break / thematic horizontal rule	
<table>	Wrapper for tabular data	<table><tr><td>1</td></tr></table>

Sample Table Markup

```

<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Marks</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Asha</td>
      <td>92</td>
    </tr>
  </tbody>
</table>

```

HTML5 — Forms and Input Elements

Item	Description	Example
<form>	Wraps a set of form controls	<form action="/submit" method="post">
<input>	A single text/data field	<input type="email" required>
type=	text, email, password, number, date, checkbox, radio, file	type="password"
<label for>	Accessible label linked to an input id	<label for="e">Email</label>
<textarea>	Multi-line text input	<textarea rows="4"></textarea>
<select><option>	Dropdown selection list	<select><option>A</option></select>
<button>	Clickable button (type=submit/button)	<button type="submit">Go</button>
placeholder	Hint text shown inside an empty input	placeholder="Enter name"
required / disabled	Validation rule / disables the field	required disabled

Sample Contact Form

```
<form action="/submit" method="post">
  <label for="name">Name</label>
  <input type="text" id="name" name="name" placeholder="Your name" required>

  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>

  <label for="msg">Message</label>
  <textarea id="msg" name="msg" rows="4"></textarea>

  <button type="submit">Send</button>
</form>
```

Accessibility (a11y) Essentials

Accessibility ensures your website can be used by everyone, including people using screen readers, keyboard-only navigation, or assistive devices. Good accessibility is also good practice for SEO and usability in general.

Item	Description	Example
alt=""	Describes an image for screen readers	
aria-label	Provides an accessible name for an element	<button aria-label="close">x</button>
role=""	Defines the ARIA role of an element	role="navigation"
tabindex	Controls keyboard focus order	tabindex="0"

Item	Description	Example
<label for>	Associates a label with an input (a11y)	<label for="n">Name</label><input id="n">
Contrast / alt text	Use sufficient colour contrast and alt text always	-

Quick Accessibility Checklist

- Every has a meaningful alt attribute (or alt="" if purely decorative).
- Every form <input> has an associated <label>.
- Interactive elements are reachable and operable using only the keyboard (Tab, Enter, Space).
- Text and background colours maintain a contrast ratio of at least 4.5:1 for normal text.
- Headings follow a logical order (h1, then h2, then h3) without skipping levels.

CSS Fundamentals — Box Model, Display and Position

Every element in CSS is treated as a rectangular box made up of content, padding, border and margin. Understanding this box model is the foundation of all CSS layout work.

The Box Model

Item	Description	Example
box-sizing	Includes padding and border inside the given width	border-box
width / height	Sets the size of the element	width: 200px; height: 100%;
margin	Outer space outside the border (T R B L or shorthand)	margin: 10px 20px;
padding	Inner space between border and content	padding: 1rem;
border	Sets width, style and colour of the border	border: 1px solid #333;
border-radius	Rounds the corners of a box	border-radius: 8px;

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

.box {
  width: 200px;
  padding: 16px;
  border: 1px solid #333;
  border-radius: 8px;
  margin: 12px auto;
}
```

Display and Positioning

Item	Description	Example
display	block / inline / inline-block / flex / grid / none	display: flex;
position	static / relative / absolute / fixed / sticky	position: absolute;
top/right/bottom/left	Offsets used together with position	top: 0; left: 0;
z-index	Controls the stacking order of overlapping boxes	z-index: 10;
overflow	visible / hidden / scroll / auto	overflow: hidden;
box-shadow	x-offset y-offset blur colour	box-shadow: 0 2px 6px #0003;

```
.card {  
  position: relative;  
}  
  
.badge {  
  position: absolute;  
  top: 8px;  
  right: 8px;  
  z-index: 10;  
}  
  
.navbar {  
  position: sticky;  
  top: 0;  
}
```

Positioning values explained

- static — default flow; top/left have no effect.
- relative — offset from its own normal position; still takes up original space.
- absolute — removed from normal flow; positioned relative to nearest positioned ancestor.
- fixed — positioned relative to the browser viewport; stays put while scrolling.
- sticky — behaves like relative until a scroll threshold, then behaves like fixed.

CSS Flexbox — Complete Guide

Flexbox is a one-dimensional layout system (row OR column) designed to distribute space and align items inside a container, even when their size is unknown or dynamic.

Item	Description	Example
display: flex	Enables flexbox on the container	display: flex;
flex-direction	row / row-reverse / column / column-reverse	flex-direction: column;
justify-content	Main-axis alignment: start/center/space-between/around	justify-content: center;
align-items	Cross-axis alignment: start/center/end/stretch	align-items: center;

Item	Description	Example
flex-wrap	nowrap / wrap	flex-wrap: wrap;
gap	Space between flex or grid items	gap: 16px;
flex: 1	Item grows/shrinks to fill available space	flex: 1;
align-self	Overrides align-items for a single item	align-self: flex-end;

Common Flexbox Patterns

Row with space between items, vertically centred

```
.row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
}
```

Equal-width flexible columns

```
.columns {
  display: flex;
  gap: 20px;
}
.columns > div {
  flex: 1;
}
```

Centering anything (both axes)

```
.center-wrapper {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

Responsive wrapping card grid

```
.card-list {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}
.card-list > .card {
  flex: 1 1 250px; /* grow shrink basis */
}
```

CSS Grid — Complete Guide

CSS Grid is a two-dimensional layout system (rows AND columns simultaneously), ideal for full page layouts and structured component grids.

Item	Description	Example
display: grid	Enables grid on the container	display: grid;
grid-template-columns	Defines column tracks	grid-template-columns: 1fr 1fr 1fr;
grid-template-rows	Defines row tracks	grid-template-rows: auto 1fr;
repeat()	Repeats a track pattern	repeat(3, 1fr)
gap	Row and column gap together	gap: 20px;
grid-column	Makes an item span multiple columns	grid-column: 1 / 3;
grid-template-areas	Named layout regions	"header header" "nav main"
place-items	Shorthand for align-items + justify-items	place-items: center;

Responsive Auto-fit Grid (most useful pattern)

```
.gallery {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 20px;  
}
```

This single rule creates a fully responsive grid: the browser automatically fits as many 220px-minimum columns as will fit, and stretches them evenly — no media queries required.

Named Template Areas Layout

```
.page {  
  display: grid;  
  grid-template-columns: 220px 1fr;  
  grid-template-rows: auto 1fr auto;  
  grid-template-areas:  
    "header header"  
    "nav    main"  
    "footer footer";  
  min-height: 100vh;  
  gap: 12px;  
}  
header { grid-area: header; }  
nav    { grid-area: nav; }  
main   { grid-area: main; }  
footer { grid-area: footer; }
```

CSS Typography and Colour

Item	Description	Example
color	Sets the text colour	color: #222;
background-color	Sets the fill colour of a box	background-color: #fafafa;
font-family	Typeface stack, with fallbacks	font-family: 'Segoe UI', sans-serif;
font-size	Size of the text	font-size: 1.125rem;
font-weight	normal / bold / 100–900	font-weight: 600;
line-height	Vertical spacing between lines of text	line-height: 1.5;
text-align	left / center / right / justify	text-align: center;
text-decoration	underline / none / line-through	text-decoration: none;
letter-spacing	Space between individual letters	letter-spacing: 0.5px;

A Simple Typographic Scale

```
h1 { font-size: 2.5rem; font-weight: 700; line-height: 1.2; }
h2 { font-size: 2rem;   font-weight: 700; line-height: 1.25; }
h3 { font-size: 1.5rem; font-weight: 600; line-height: 1.3; }
p  { font-size: 1rem;   font-weight: 400; line-height: 1.6; }
small { font-size: 0.875rem; color: #666; }
```

Colour Notes

- Hex: #rrggbb (e.g. #1a5276) — most common format.
- RGB / RGBA: rgba(26, 82, 118, 0.8) — the 4th value controls opacity.
- HSL: hsl(210, 65%, 28%) — easier to reason about lightness and saturation.
- Keep a limited palette: one primary colour, one accent colour, and 2–3 neutral greys.

Units, Responsive Design and States

Item	Description	Example
px / %	Fixed pixels / relative percentage	width: 50%;
rem / em	Root-relative / parent-relative units	font-size: 1.5rem;
vw / vh	Percentage of viewport width / height	height: 100vh;
transition	Animates a property change smoothly	transition: all 0.3s ease;
transform	translate / scale / rotate an element	transform: translateY(-4px);
@media	Defines a responsive breakpoint (mobile-first)	@media (min-width: 768px) { ... }
:hover / :focus	Pseudo-class states	button:hover { opacity:.8; }
::before / ::after	Pseudo-elements; insert generated content	content: '★';

Mobile-First Media Query Pattern

```
/* base styles = mobile, written first */
.card {
  width: 100%;
}

@media (min-width: 768px) {
  .card { width: 50%; }
}

@media (min-width: 1024px) {
  .card { width: 33.33%; }
}
```

Common Breakpoints

Breakpoint	Typical device	min-width
sm	Large phones	480px
md	Tablets	768px
lg	Laptops	1024px
xl	Desktops	1280px

Smooth Hover Transition Example

```
.btn {
  background: #1a5276;
  transform: translateY(0);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.btn:hover {
  transform: translateY(-2px);
  box-shadow: 0 6px 14px rgba(0,0,0,0.2);
}
```

Pseudo-classes and Pseudo-elements

Pseudo-classes target an element in a particular state; pseudo-elements target a specific part of an element or insert generated content.

Common Pseudo-classes

Pseudo-class	Meaning
:hover	While the pointer is over the element
:focus	While the element has keyboard focus
:active	While the element is being clicked/pressed

Pseudo-class	Meaning
:first-child / :last-child	First or last child of its parent
:nth-child(n)	Matches the nth child (e.g. even, odd, 3n)
:not(selector)	Matches elements that do NOT match the selector
:disabled / :checked	Form state — disabled input / checked checkbox
:visited	A link the user has already visited

Common Pseudo-elements

Pseudo-element	Meaning
::before	Inserts generated content before the element's content
::after	Inserts generated content after the element's content
::placeholder	Styles the placeholder text of an input
::selection	Styles text highlighted/selected by the user
::first-letter	Styles the first letter of a block of text

Practical Examples

```
/* Zebra-striped table rows */
tr: nth-child(even) { background: #f5f5f5; };

/* Required-field asterisk using ::after */
label.required::after {
  content: " *";
  color: #922b21;
}

/* Custom placeholder colour */
input::placeholder {
  color: #999;
  font-style: italic;
}

/* Disable styling for disabled buttons */
button.disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
```

Working with Images in HTML and CSS

There are two ways to add an image to a web page: the HTML `<img>` tag (content images) and the CSS `background-image` property (decorative images). Choosing the right one matters for accessibility and layout control.

HTML Tag — content images

```

```

- Use for meaningful images: photos, diagrams, logos that convey information.
- Always include a descriptive alt attribute for accessibility and SEO.
- The image participates in the document flow like any other content.

CSS background-image — decorative images

```
.hero {  
  background-image: url("banner.jpg");  
  background-size: cover;  
  background-position: center;  
  background-repeat: no-repeat;  
  height: 400px;  
}
```

- Use for purely decorative backgrounds, patterns or hero sections.
- background-size: cover fills the box while preserving aspect ratio (may crop).
- background-size: contain fits the whole image inside the box (may letterbox).

Controlling Sizing with object-fit

```
.avatar {  
  width: 80px;  
  height: 80px;  
  border-radius: 50%;  
  object-fit: cover; /* crops to fill the box neatly */  
}
```

Responsive Images

```
img {  
  max-width: 100%;  
  height: auto;  
  display: block;  
}
```

Alignment and Centering Techniques

Centering elements is one of the most common tasks in CSS. Below are the most reliable modern methods.

Centre a Block Horizontally (auto margins)

```
.container {  
  width: 800px;  
  max-width: 90%;  
  margin: 0 auto;  
}
```

Centre Anything — Flexbox (most common)

```
.center {  
  display: flex;  
  justify-content: center; /* horizontal */  
  align-items: center;     /* vertical */  
  min-height: 100vh;  
}
```

Centre Anything — Grid

```
.center {  
  display: grid;  
  place-items: center;  
  min-height: 100vh;  
}
```

Centre with Absolute Positioning

```
.center {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  transform: translate(-50%, -50%);  
}
```

Text Alignment

```
.text-center { text-align: center; }  
.text-right { text-align: right; }
```

Spacing — Margin, Padding and Gap

Correct spacing is what makes a layout feel clean and professional. Understanding the difference between margin, padding and gap avoids most common layout bugs.

Property	Space location	Notes
padding	Inside the border, around the content	Increases the element's visual size unless box-sizing: border-box is set
margin	Outside the border, between elements	Adjacent vertical margins can 'collapse' into the larger value
gap	Between flex or grid items only	Does not add space around the outer edge of the container

Shorthand Order (clockwise from top)

```
margin: 10px;           /* all sides */  
margin: 10px 20px;      /* vertical horizontal */  
margin: 10px 20px 15px 5px; /* top right bottom left */
```

Consistent Spacing Scale (recommended)

```
:root {  
  --space-1: 4px;  
  --space-2: 8px;  
  --space-3: 16px;  
  --space-4: 24px;  
  --space-5: 32px;  
}  
.card { padding: var(--space-3); margin-bottom: var(--space-4); }
```

UI Components — Buttons

A well-designed button communicates its purpose and state clearly through colour, spacing and feedback on hover/focus/active/disabled states.

Basic Reusable Button

```
<button class="btn btn-primary">Submit</button>
```

```
.btn {  
  display: inline-block;  
  padding: 10px 20px;  
  font-size: 1rem;  
  font-weight: 600;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
  transition: background 0.2s ease, transform 0.15s ease;  
}  
  
.btn-primary {  
  background: #1a5276;  
  color: #ffffff;  
}  
.btn-primary:hover { background: #154360; }  
.btn-primary:active { transform: scale(0.97); }  
  
.btn-outline {  
  background: transparent;  
  color: #1a5276;  
  border: 2px solid #1a5276;  
}  
  
.btn:disabled {  
  opacity: 0.5;  
  cursor: not-allowed;  
}
```

Button Variants Checklist

- Primary — solid background, used for the main call-to-action.
- Secondary / Outline — border only, used for lower-priority actions.
- Ghost / Text — no border or background, used for minor actions.
- Always style :hover, :focus-visible, :active and :disabled states.

UI Components — Cards

A card groups related content (image, title, description, action) inside a bordered, elevated box. Cards are one of the most reused UI patterns on the web.

```
<div class="card">
  
  <div class="card-body">
    <h3 class="card-title">Product Name</h3>
    <p class="card-text">Short description of the product goes here.</p>
    <button class="btn btn-primary">Buy Now</button>
  </div>
</div>
```

```
.card {
  max-width: 320px;
  background: #ffffff;
  border: 1px solid #e0e0e0;
  border-radius: 10px;
  overflow: hidden;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  transition: box-shadow 0.25s ease, transform 0.25s ease;
}

.card:hover {
  box-shadow: 0 8px 20px rgba(0,0,0,0.15);
  transform: translateY(-4px);
}

.card-img {
  width: 100%;
  height: 180px;
  object-fit: cover;
}

.card-body {
  padding: 16px;
}

.card-title {
  margin-bottom: 8px;
  font-size: 1.1rem;
}

.card-text {
  color: #555;
  margin-bottom: 12px;
}
```

UI Components — Navigation Bars

```
<header class="navbar">
  <a href="#" class="logo">Brand</a>
  <nav class="nav-links">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </nav>
</header>
```

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 14px 24px;
  background: #ffffff;
  border-bottom: 1px solid #e0e0e0;
  position: sticky;
  top: 0;
  z-index: 100;
}
.nav-links {
  display: flex;
  gap: 20px;
}
.nav-links a {
  text-decoration: none;
  color: #222;
  font-weight: 500;
}
.nav-links a:hover {
  color: #1a5276;
}

/* collapse to column on small screens */
@media (max-width: 600px) {
  .navbar { flex-direction: column; gap: 10px; }
}
```

UI Components — Forms and Input Styling

```
<form class="form">
  <div class="form-group">
    <label for="email">Email</label>
    <input type="email" id="email" placeholder="you@example.com">
  </div>
  <button type="submit" class="btn btn-primary">Sign Up</button>
</form>
```

```

.form {
  display: flex;
  flex-direction: column;
  gap: 16px;
  max-width: 360px;
}
.form-group {
  display: flex;
  flex-direction: column;
  gap: 6px;
}
.form-group label {
  font-weight: 600;
  font-size: 0.9rem;
}
.form-group input,
.form-group textarea,
.form-group select {
  padding: 10px 12px;
  border: 1px solid #ccc;
  border-radius: 6px;
  font-size: 1rem;
  transition: border-color 0.2s ease;
}
.form-group input:focus {
  outline: none;
  border-color: #1a5276;
  box-shadow: 0 0 0 3px rgba(26,82,118,0.15);
}

```

UI Components — Boxes, Borders and Shadows

Border Styles

```

.solid { border: 2px solid #333; }
.dashed { border: 2px dashed #333; }
.dotted { border: 2px dotted #333; }
.rounded-full { border-radius: 999px; } /* pill shape */

```

Elevation with Shadows (small to large)

```

.shadow-sm { box-shadow: 0 1px 3px rgba(0,0,0,0.10); }
.shadow-md { box-shadow: 0 4px 10px rgba(0,0,0,0.12); }
.shadow-lg { box-shadow: 0 12px 28px rgba(0,0,0,0.18); }

```

Alert / Notice Box

```

.alert {
  padding: 12px 16px;
  border-left: 4px solid #1a5276;
  background: #eaf2f8;
  border-radius: 4px;
}

```

```

}
.alert-error {
  border-left-color: #922b21;
  background: #fdecea;
}

```

Tailwind CSS — Complete Reference

Tailwind is a utility-first CSS framework: instead of writing custom CSS classes, you compose small single-purpose utility classes directly in your HTML. This speeds up development considerably once the class names are familiar.

CDN Setup (no build step required)

```
<script src="https://cdn.tailwindcss.com"></script>
```

Utility Reference

Item	Description	Example
p-4 / px-4 / py-2	Padding (all sides / x-axis / y-axis)	<div class="p-4">
m-4 / mx-auto	Margin / centre a block horizontally	<div class="mx-auto">
w-full / h-screen	Width 100% / height 100vh	<div class="h-screen w-full">
flex / grid	Enable flex or grid display	<div class="flex gap-4">
justify-center / items-center	Flex alignment utilities	<div class="flex justify-center items-center">
grid-cols-3 / gap-4	Grid columns and spacing	<div class="grid grid-cols-3 gap-4">
text-xl / font-bold	Font size / font weight	<h1 class="text-2xl font-bold">
bg-blue-500 / text-white	Background / text colour, shades 50–900	<button class="bg-blue-500 text-white">
rounded-lg / shadow-md	Border radius / box shadow	<div class="rounded-lg shadow-md">
hover: / focus:	State variant prefixes	<button class="hover:bg-blue-700">
sm: md: lg: xl:	Responsive breakpoint prefixes (mobile-first)	<div class="grid md:grid-cols-2 lg:grid-cols-3">
space-y-4	Vertical spacing between child elements	<div class="space-y-4">
transition / duration-300	Animate utility value changes	<div class="transition duration-300">

Example Card Component in Tailwind

```

<div class="max-w-sm mx-auto p-6 bg-white
  rounded-lg shadow-md hover:shadow-xl

```

```
transition duration-300">
<h2 class="text-xl font-bold mb-2">Title</h2>
<p class="text-gray-600">Body text</p>
</div>
```

Example Responsive Navbar in Tailwind

```
<header class="flex justify-between items-center
px-6 py-4 bg-white shadow-sm sticky top-0">
<span class="font-bold text-lg">Brand</span>
<nav class="hidden md:flex gap-6">
  <a href="#" class="hover:text-blue-600">Home</a>
  <a href="#" class="hover:text-blue-600">About</a>
</nav>
</header>
```

Tip: hidden md:flex hides an element on mobile and shows it as flex from the md breakpoint upward — a very common responsive-navbar pattern.

Responsive Design Patterns

Fluid Typography with clamp()

```
h1 {
  font-size: clamp(1.75rem, 4vw + 1rem, 3rem);
}
```

clamp(min, preferred, max) lets font size scale smoothly between a minimum and maximum value across all screen sizes, without needing multiple media queries.

Responsive Grid Without Media Queries

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 16px;
}
```

Hiding / Showing Elements per Breakpoint

```
.mobile-only { display: block; }
.desktop-only { display: none; }

@media (min-width: 768px) {
  .mobile-only { display: none; }
  .desktop-only { display: block; }
}
```

Container Widths

```
.container {  
  width: 100%;  
  max-width: 1140px;  
  margin-inline: auto;  
  padding-inline: 16px;  
}
```

CSS Best Practices and Design Tips

- Start with a CSS reset and box-sizing: border-box on every project.
- Use a mobile-first approach: write base styles for small screens, then add min-width media queries.
- Keep a limited colour palette and a consistent spacing scale (see Chapter 15).
- Prefer rem for font sizes so text respects the user's browser settings.
- Use Flexbox for one-dimensional alignment; use Grid for two-dimensional page layout.
- Name classes by purpose, not appearance — e.g. .btn-primary, not .blue-button.
- Avoid deeply nested selectors; keep specificity low and predictable.
- Use CSS custom properties (--variables) for colours and spacing so themes are easy to change.
- Always test layouts at common breakpoints: 375px, 768px, 1024px, 1440px.
- Optimise images (compressed formats, correct dimensions) to keep pages fast.

CSS Custom Properties Example

```
:root {  
  --color-primary: #1a5276;  
  --color-text: #222222;  
  --radius: 8px;  
}  
.btn {  
  background: var(--color-primary);  
  border-radius: var(--radius);  
}
```

Visual Studio Code — Setup and Productivity

Visual Studio Code (VS Code) is the most widely used code editor for frontend development. The tips below will significantly speed up HTML/CSS workflow.

Essential Extensions

Extension	Purpose
Live Server	Launches a local server with auto page-refresh on save
Prettier	Automatically formats HTML, CSS and JS code consistently
Auto Rename Tag	Renames the matching closing tag automatically
HTML CSS Support	Autocomplete for class names and CSS properties
Tailwind CSS IntelliSense	Autocomplete, linting and hover previews for Tailwind classes

Extension	Purpose
Color Highlight	Shows a colour preview next to hex/RGB values in code
Path Intellisense	Autocompletes file paths for links, images and scripts

Emmet Abbreviations (built into VS Code)

Emmet lets you type a short abbreviation and press Tab to expand it into full HTML/CSS. It is built into VS Code by default.

Abbreviation	Expands to
!	Full HTML5 boilerplate
div.card	<div class="card"></div>
ul>li*3	A containing 3 items
nav>a*3	A <nav> containing 3 <a> links
div.row>div.col*3	A row div containing three column divs
m10	margin: 10px; (in a CSS file)
df	display: flex; (in a CSS file)

Useful Keyboard Shortcuts

Shortcut (Windows/Linux)	Action
Ctrl + `	Toggle the integrated terminal
Ctrl + P	Quickly open a file by name
Ctrl + Shift + P	Open the Command Palette
Alt + Shift + F	Format the current document (with Prettier)
Ctrl + D	Select the next occurrence of the current word
Ctrl + /	Toggle line comment
Alt + Click	Place multiple cursors for simultaneous editing
Ctrl + Shift + L	Select all occurrences of the current selection

Recommended Settings

- Enable Format on Save so Prettier tidies code automatically.
- Enable Auto Save (files > autoSave: onFocusChange) to avoid losing work.
- Use the built-in split editor (Ctrl + \) to view HTML and CSS side by side.
- Right-click any HTML file and choose 'Open with Live Server' to preview instantly in the browser.

Consolidated Quick-Reference Tables

Flexbox at a Glance

Property	Values
display	flex
flex-direction	row row-reverse column column-reverse
justify-content	flex-start center flex-end space-between space-around space-evenly
align-items	stretch flex-start center flex-end baseline
flex-wrap	nowrap wrap wrap-reverse
flex-grow / flex-shrink / flex-basis	flex: <grow> <shrink> <basis>;

Grid at a Glance

Property	Values
display	grid
grid-template-columns	e.g. repeat(3, 1fr) 200px 1fr repeat(auto-fit, minmax(200px, 1fr))
grid-template-rows	e.g. auto 1fr auto
grid-column / grid-row	start / end, e.g. 1 / 3
justify-items / align-items	start center end stretch
place-items	align-items justify-items shorthand

Colour & Box Shadow Cheat Values

Use case	Suggested value
Subtle card shadow	0 2px 8px rgba(0,0,0,0.08)
Hover shadow	0 8px 20px rgba(0,0,0,0.15)
Border colour (light)	#e0e0e0
Text colour (body)	#222222
Muted text colour	#666666 / #999999

Common Selector Syntax

Selector	Meaning
.class	Selects all elements with that class
#id	Selects the single element with that id
div p	Selects <p> anywhere inside a <div> (descendant)
div > p	Selects <p> that is a direct child of <div>
a, button	Selects both <a> and <button> elements
div.card.featured	Selects a div with BOTH classes